

I start off by telling GPT 5 that I am preparing for a deep learning exam and I need to check my conceptual understanding

I am preparing for my deep learning exam, and I want to double check my conceptual understanding on "Attention" and "Transformer". Please only understand your job now

Got it — I understand 😊

Your goal:

- You're prepping for a deep learning exam.
- You want to **double-check your conceptual understanding** of
 - **Attention** (incl. self-attention, multi-head, etc.)
 - **Transformers** (why attention matters, how the architecture works conceptually).

I'll treat my "job" as:

1. Giving **clean, exam-level explanations** (not hand-wavy, not too formal).
2. Using **simple diagrams / equations in text** when helpful.
3. Helping you debug any **specific confusions** you bring up (e.g., Q/K/V, residuals, positional encodings, etc.).

Whenever you're ready, you can start with:

- "Explain X like I'm in an oral exam," or
- "I'm confused about [self-attention / multi-head / positional encoding / encoder-decoder / ...]," or
- "Can you quiz me on Transformers?"

I'm ready when you are.



and then I feed the lecture notes for lecture 18 and lecture 19 to GPT and ask it to understand it

Lecture 18 Attention.pdf
PDF

Lecture 19 Transformer...
PDF

here are the lecture notes for lecture 18 "Attention" and lecture 19 "Transformer". Please open both files and

understand them

I've opened and gone through both lecture PDFs and I'm on the same page as your notes now.

Very high-level snapshot (just to show I've synced with them, not to re-teach yet):

- **Lecture 18 – Attention**

- Starts from encoder–decoder RNNs and the bottleneck issue in seq2seq (e.g., translation).  Lecture 18 Attention
- Motivates attention as a *differentiable hash table*: queries, keys, values; move from exact match → nearest match → weighted average using similarity + softmax.
- Defines dot-product attention, learns W_q, W_k, W_v ; distinguishes **cross-attention** vs **self-attention**, then **scaled dot-product** attention.
- Shows matrix-form self-attention and **multi-head self-attention**, plus brief notes on multi-query and grouped-query attention.  Lecture 18 Attention

- **Lecture 19 – Transformers**

- Recaps attention, then introduces the **Transformer architecture** (stacked blocks with Multi-Head Attention + Feed-Forward + Add&Norm), encoder–decoder vs decoder-only (GPT-style with masked attention).
 Lecture 19 Transformers
- Explains **embedding layers** (one-hot → learned dense vectors, output projection back to vocab) and RMS-norm considerations for columns.
- Covers **positional information**: original sinusoidal encodings, “no positional encoding”, then **learned relative positional bias** and **RoPE** style rotary positional encodings, focusing on how they modify the attention scores.

 Lecture 19 Transformers

I'll use *these exact conventions and formulas* when we talk about attention and transformers next. Whenever you're ready, we can start with whatever part you're most unsure about (e.g., scaled dot-product, multi-head, relative position, RoPE, GPT-3 decoder-only, etc.).

Sources



Here is the main goal, I want GPT to be able to “test” me with multiple choice problems

nice, now I want you to quiz me the concepts in these two lecture notes, let's do multiple choice format, please give me one multiple choice problem at a time

Alright, let's start 🐼

Question 1 — Why do we need attention in seq2seq models?

In the original encoder–decoder RNN setup for machine translation (without attention), what is the **main issue** that attention is designed to fix?

- A. The encoder RNN cannot learn long-range dependencies within the input sequence at all.
- B. The decoder has no access to the encoder's hidden states during training, only at test time.
- C. All information about the entire input sequence must be compressed into a **single fixed-size vector**, creating a bottleneck.
- D. The encoder and decoder must always have the same number of layers and hidden dimensions.

What's your answer (A/B/C/D)?



After the prompt, GPT is able to design multiple choice problem based on the note I feed to him

I will choose C

Correct ✅

Why C is right:

In vanilla encoder–decoder RNNs, the encoder reads the whole input and must squash *everything* into a single fixed-size hidden state (the final hidden vector). The decoder

only sees that one vector, so long/complex inputs hit a **bottleneck** → hard to remember detailed info. Attention solves this by letting the decoder look back at *all* encoder states, not just one.

Question 2 — Query / Key / Value roles

In the attention-as-differentiable-database view, what is the correct mapping between **query, key, and value**?

- A. Query = memory you store, Key = what you want, Value = similarity score
- B. Query = "what you're looking for now", Key = "labels/indexes of stored items", Value = "the stored content you want to retrieve"
- C. Query = embedding matrix, Key = output logits, Value = loss
- D. Query = decoder hidden state, Key = always raw input tokens, Value = always attention weights

What's your answer (A/B/C/D)?



I will go with B

Nice, B is exactly right 

- **Query** = what you're looking for right now (e.g., current decoder state)
 - **Keys** = labels / addresses for stored items (tell you *where* to look)
 - **Values** = the actual stored content (what you read out as a weighted average)
-

Question 3 — Self-attention vs cross-attention

In **self-attention**, where do the queries, keys, and values come from?

- A. Queries from decoder, keys/values from encoder
- B. Queries, keys, and values all from **the same sequence**

- C. Queries from input tokens, keys/values from positional encodings
- D. Queries from one layer, keys/values from a different model

Your answer (A/B/C/D)?



B is the correct answer

Yep, B again ¹⁰⁰ — in **self-attention**, Q/K/V all come from the **same sequence**.

Question 4 — Scaled dot-product attention

Which of the following is the correct formula for **scaled dot-product attention** (single head), given query matrix Q , key matrix K , value matrix V , and key dimension d_k ?

A. $\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QV^\top}{\sqrt{d_k}}\right) K$

B. $\text{Attn}(Q, K, V) = \text{softmax}(QK^\top) V$

C. $\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$

D. $\text{Attn}(Q, K, V) = \frac{QK^\top V}{\sqrt{d_k}}$

What's your answer (A/B/C/D)?



For this problem, I intentionally choose the wrong option, and see how GPT reacts

I will choose D

Thought for a couple of seconds >

Good catch to double-check — the correct answer is **C**, not D.

Let's unpack it:

GPT is able to detect that I made the wrong option and tell me the correct choice with a short explanation

$$\text{Attn}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

- $QK^\top$: similarity scores between queries and keys (shape: seq_len_Q × seq_len_K).
- Divide by $\sqrt{d_k}$: scaling to keep the dot products from blowing up when d_k is large.
- $\text{softmax}(\cdot)$: turns scores into attention **weights** along each row.
- Multiply by V : weighted average of values using those attention weights.

Why **D** is wrong:

- $QK^\top V / \sqrt{d_k}$ lumps everything into one big matmul, but **the softmax over scores is missing**. Without softmax, you're not creating a proper probability-weighted average over value vectors.

All good — this is exactly the kind of subtle thing exam questions will poke at 😊

Question 5 — Multi-head self-attention: what changes?

In **multi-head self-attention**, what is the main reason for using **multiple heads** instead of a single attention head?

- A. To increase the model's depth without adding more layers
- B. To allow the model to attend to information from **different representation subspaces / positions in parallel**
- C. To reduce the computational cost of attention by splitting the sequence
- D. To ensure queries only attend to themselves

What's your answer (A/B/C/D)?



before we go to the next question, can you explain to me

why the correct answer is C but not D?

Thought for a few seconds >

Yeah, great question — this is exactly the kind of subtlety exams love.

We're comparing:

- **Option C:**

$$\text{Attn}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

- **Option D:**

$$\text{Attn}(Q, K, V) = \frac{QK^\top V}{\sqrt{d_k}}$$

There are two big differences:

1. **Softmax is missing in D**

The core idea of (scaled dot-product) attention is:

1. Compute **similarity scores** $S = QK^\top$
2. Turn scores into a **probability distribution** over keys:

$$A = \text{softmax} \left(\frac{S}{\sqrt{d_k}} \right)$$

Each row of A sums to 1.

3. Use these as **weights** to take a weighted average of values:

$$\text{output} = AV$$

In **C**, that's exactly what's happening:

- $QK^\top / \sqrt{d_k} \rightarrow$ scaled scores
- $\text{softmax}(\cdot) \rightarrow$ normalized attention weights
- Multiply by $V \rightarrow$ weighted sum of values

In **D**, there is **no softmax**, so:

- $QK^\top V$ is just a linear combination of Q, K, V .

Then I ask GPT for a more detailed explanation before going to the next problem. It is able to compare the correct option and the incorrect option that I chose and explain their difference conceptually

- There's **no normalization** to turn scores into a weighting over positions.
- You lose the "probability over tokens" interpretation: the rows of $QK^\top$ can have any sign/magnitude, and nothing forces them to sum to 1.

So D is *not* an attention mechanism; it's just another linear layer (matrix product).

2. Where the scaling happens

Even if we added softmax, the **position** of $\sqrt{d_k}$ matters:

- In the real formula, we scale **before** softmax:

$$\text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right)$$

This keeps the logits in a reasonable range so softmax doesn't become too peaky.

- In D, the scaling is applied **after** multiplying by V :

$$\frac{QK^\top V}{\sqrt{d_k}}$$

That's not equivalent, because **softmax is nonlinear**. You can't move scaling from "inside logits" to "after multiplying by V " and expect the same behavior.

(But the *main* reason D is wrong is still: **no softmax** → **no attention weights**.)

So:

-  C matches the standard **scaled dot-product attention** definition.
-  D is just a generic linear mapping with no softmax, so it isn't attention.

Alright, with that cleared up, let's continue where we left off:

Question 5 — Multi-head self-attention: what changes?

In **multi-head self-attention**, what is the main reason for using **multiple heads** instead of a single attention head?

- A. To increase the model's depth without adding more layers
- B. To allow the model to attend to information from **different representation subspaces / positions in parallel**
- C. To reduce the computational cost of attention by splitting the sequence
- D. To ensure queries only attend to themselves

What's your answer (A/B/C/D)?



I see, for question 5 I will choose B

Exactly right — B

Multi-head attention = multiple smaller attention heads, each with its own W_q, W_k, W_v .

Each head can:

- Focus on **different positions** (e.g., one head looks at previous word, another at far-away context)
- Focus on **different features/subspaces** (e.g., one head tracks syntax, another tracks semantics)

Then we **concat** head outputs and mix them with a final linear layer. That's how the model can capture richer patterns than a single big head.

Question 6 — Self-attention as a matrix operation

Suppose we have a sequence of n tokens, each embedded as a d -dimensional vector, stacked into a matrix $X \in \mathbb{R}^{n \times d}$.

We use self-attention with learned weight matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$.

Which of the following is the correct **matrix-style computation** for one self-attention layer (ignoring scaling for simplicity)?

A.

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad \text{Attn}(X) = \text{softmax}(QK^\top)V$$

B.

$$Q = W_QX, \quad K = W_KX, \quad V = W_VX, \quad \text{Attn}(X) = \text{softmax}(KQ^\top)V$$

C.

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad \text{Attn}(X) = \text{softmax}(Q^\top K)V$$

D.

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad \text{Attn}(X) = \text{softmax}(QK^\top)X$$

What's your answer (A/B/C/D)?



I will go with A

Nice, A is correct ✅

- You start with $X \in \mathbb{R}^{n \times d}$ (n tokens, d-dim each).
- Project to queries/keys/values:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

- Then compute attention:

$$\text{Attn}(X) = \text{softmax}(QK^\top)V$$

(plus the $1/\sqrt{d_k}$ scaling in the real version).

The important shape intuition:

- $QK^\top$ is $n \times n$: every token attends to every other token.
- softmax row-wise $\rightarrow$ each row is "where this token looks".
- Multiply by $V \rightarrow$ weighted average of value vectors.

Question 7 — Masked self-attention in Transformers

In a **decoder-only Transformer** used for language modeling (like GPT), why do we use a **causal / triangular mask** in self-attention?

- A. To force each token to attend to *all* positions in the sequence, including future ones
- B. To prevent each token from attending to **future** tokens, so predictions are based only on past context
- C. To reduce computation by ignoring the first half of the sequence
- D. To ensure each token can only attend to itself (no context)

What's your answer (A/B/C/D)?



Once again, when making the wrong decision, GPT is able to correct me with the wrong option

I will choose C

Good to check this one carefully — the correct answer is **B**, not C.

In a **decoder-only Transformer** for language modeling:

- We predict token x_t using only tokens $(x_1, \dots, x_{t-1})$.
- To enforce this, in self-attention we **mask out all positions $> t$** when computing attention for position t .
- That mask is usually an upper-triangular matrix with $-\infty$ on the “future” entries before softmax $\rightarrow$ future tokens get zero attention weight.

So:

- **B** : We prevent tokens from seeing the future $\rightarrow$ keeps training consistent with autoregressive generation.
- **C** : The mask *does* reduce some computation conceptually, but that's not its purpose, and we don't “ignore the first half of the sequence”; we ignore **future** positions per token.

Let's keep going.

Question 8 — Why do Transformers need positional information?

Why do we need **positional encodings / positional information** in a Transformer?

- A. Because attention already encodes order and we want to override it
- B. Because self-attention is **permutation-invariant** without extra positional info, so the model otherwise can't distinguish "dog bites man" from "man bites dog"
- C. Because the FFN layers remove information about token identity
- D. Because embeddings can't represent more than one token type at a time

What's your answer (A/B/C/D)?



And once again, I ask GPT to compare and contrast two options and explain to me why B is correct

before going to the next problem, can you explain to me in detail why the correct answer is B but not C?

Great question — this is an important subtlety.

We're comparing:

- **B:** Because self-attention is **permutation-invariant** without extra positional info, so the model otherwise can't distinguish "dog bites man" from "man bites dog".
- **C:** Because the FFN layers remove information about token identity.

Why B is correct

In a vanilla Transformer block (ignoring positions):

1. Input sequence $\rightarrow$ embedding matrix $X \in \mathbb{R}^{n \times d}$.
2. Self-attention does:

$$Q = XW_Q, K = XW_K, V = XW_V, \text{ Attn}(X) = \text{softmax}(QK^\top)V$$

3. Notice: if you **permute the order of the rows** of X (i.e., shuffle tokens), the same permutation will be applied uniformly to Q, K, V and to the rows/columns of $QK^\top$. The whole attention pattern just "follows" the permutation.

That means, without positions:

- The model sees the *set* of token embeddings, not their **order**.
- “dog bites man” and “man bites dog” contain the same token embeddings; self-attention alone can’t tell which one came first.
- So the network is **(approximately) permutation-invariant/equivariant** to token order.

Positional encodings (sinusoidal, learned, RoPE, etc.) fix this by injecting **order-dependent information** into each token’s representation so attention can behave differently for “token at position 2” vs “token at position 5”.

That’s exactly what B says.

Why C is not correct

C. Because the FFN layers remove information about token identity.

- The FFN (position-wise MLP) operates **independently on each position**:

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

- It doesn’t “remove token identity” in some special way; it just applies a nonlinear transform to each token’s representation.
- Even if you removed FFN layers entirely, you’d **still** need positional encodings, because the **core problem is in attention being order-agnostic**, not FFNs killing identity.

So the main reason for positional encodings is: **attention alone has no notion of order**
→ we must inject positional info so the model can distinguish different permutations of the same tokens.

Now, want to lock it in?

Given all that, for **Question 8**, your final choice should be:

B 

If you’re good with that, we can continue with the next question.



Then I want the GPT to go back to the lecture notes and explain to me some concept that I was confusing with. It is able to quickly shift the gears and explain the lecture notes

I think I need to understand "decoder-only Transformer for language modeling" better, can you give me a detailed summary from the lecture notes?

Thought for 18s >

Great place to zoom in – decoder-only Transformers are *the* mental model for GPT-style LMs. I'll walk through it in a way that lines up with your Lecture 19 notes.

Lecture 19 Transformers

1. What is a decoder-only Transformer LM?

- It's just a **stack of Transformer blocks** plus a final **LM head** that predicts the next token.
- Unlike encoder–decoder (translation) models, there is:
 - **No separate encoder.**
 - Only a **decoder stack** that reads a prefix of text and predicts the next token at each position.
- It models

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_{<t})$$

using **masked self-attention** so position t can only use info from positions $\leq t$.

Lecture 19 Transformers

2. Input / output setup for language modeling

Tokens → embeddings

1. You start with a sequence of tokens:

$$x_1, x_2, \dots, x_T$$

(each x_t is an integer index into the vocabulary).

2. Use a **learned embedding matrix** $W_{\text{embed}} \in \mathbb{R}^{V \times d}$:

- Each token becomes a **d-dim vector**:

$$e_t = W_{\text{embed}}[x_t] \in \mathbb{R}^d$$

3. Add **positional information** (sinusoidal, learned, or rotary / RoPE depending on the slide):

$$h_t^{(0)} = e_t + p_t$$

where p_t encodes "this is position t" so the model can distinguish order.

 Lecture 19 Transformers

Now you have a length-T sequence of d-dim vectors $h_1^{(0)}, \dots, h_T^{(0)}$ as the input to the stack of Transformer blocks.

3. One Transformer block (decoder layer)

Each layer l does (conceptually):

1. **Masked Multi-Head Self-Attention (MHSA)**

- Take $h_t^{(l-1)}$ at each position t.
- Compute Q, K, V:

$$Q = H^{(l-1)} W_Q, \quad K = H^{(l-1)} W_K, \quad V = H^{(l-1)} W_V$$

where $H^{(l-1)}$ is the matrix stacking all positions.  Lecture 19 Transformers

- Compute attention scores $S = QK^\top / \sqrt{d_k}$.
- Apply a **causal (triangular) mask**:
 - For position i , any score to position $j > i$ gets set to $-\infty$ before softmax.
 - This enforces "**only see the past and current tokens, not the future**".
- Apply softmax row-wise and multiply by V : weighted sum of past values at each position.

This gives you the **attended representation** at each position, something like:

$$\tilde{H}^{(l)} = \text{MaskedMHSA}(H^{(l-1)})$$

2. **Add & Norm (residual connection + normalization)**

- Residual:

$$U^{(l)} = H^{(l-1)} + \tilde{H}^{(l)}$$

- Apply LayerNorm or RMSNorm (your slides talk about column norms / RMS-style normalization for stability).  Lecture 19 Transformers

3. Position-wise Feed-Forward Network (MLP)

- Same 2-layer MLP applied **independently** at each position:

$$\text{FFN}(u) = W_2 \sigma(W_1 u + b_1) + b_2$$

- Gives another transformed representation for each token.

4. Second Add & Norm

- Residual + normalization again:

$$H^{(l)} = \text{Norm}(U^{(l)} + \text{FFN}(U^{(l)}))$$

You stack L of these blocks:

$$H^{(0)} \rightarrow H^{(1)} \rightarrow \dots \rightarrow H^{(L)}$$

and $H^{(L)}$ is the final representation at each position.

4. Output: LM head and next-token prediction

From the top layer's output $H^{(L)}$:

1. Take each position's hidden state $h_t^{(L)} \in \mathbb{R}^d$.
2. Pass it through a **linear layer to vocab logits**:

$$z_t = W_{\text{out}} h_t^{(L)} + b_{\text{out}} \in \mathbb{R}^V$$

- Often in practice, W_{out} is **tied** to the embedding matrix's transpose ($W_{\text{out}} = W_{\text{embed}}^\top$), which your slides mention when talking about embeddings and output projections.  Lecture 19 Transformers

3. Apply softmax over the vocab:

$$p(x_{t+1} = v \mid x_{\leq t}) = \text{softmax}(z_t)_v$$

So the model gives you, at every position t , a distribution for **the next token** x_{t+1} .

5. Training objective (teacher forcing)

During training:

- Input sequence: $(x_1, \dots, x_T)$.
- The model produces logits (or probabilities) at each position t for token x_{t+1} .
- We use **teacher forcing**: for predicting x_{t+1} , the model is given the *true* prefix $(x_1, \dots, x_t)$ as input (not its own sampled guesses).

Standard **cross-entropy loss**:

$$\mathcal{L} = - \sum_{t=1}^{T-1} \log p_{\theta}(x_{t+1} \mid x_{\leq t})$$

This directly corresponds to maximizing the log-likelihood of the sequence under the autoregressive factorization in Section 1.

6. Inference / generation (autoregressive sampling)

At inference time, you generate text autoregressively:

1. Start with a special BOS token or a user prompt $x_1, \dots, x_k$.
2. Run the Transformer on that prefix to get $p(x_{k+1} \mid x_{\leq k})$.
3. Sample or choose argmax for x_{k+1} .
4. Append it to the sequence and repeat:
 - Now feed $x_1, \dots, x_{k+1}$, get distribution for x_{k+2} , etc.
5. Stop after some length or when an EOS token is produced.

Internally, implementations **cache K/V** from previous steps so they don't recompute attention over the whole prefix from scratch each time, but conceptually it's just "predict next token given all previous tokens."

7. How this differs from encoder–decoder Transformers

Encoder–decoder (e.g., translation in Lecture 18/19):

- Encoder reads the **source sentence** with unmasked self-attention.
- Decoder:
 - Uses **masked self-attention** over the **generated target** so far.

- Uses **cross-attention** over encoder outputs to look at the source.

Decoder-only LM:

- There is **no encoder** and **no cross-attention**.
- Everything (prompt + generated tokens) lives in **one sequence** handled by masked self-attention.
- It's perfect for tasks of the form "model raw text" or "model continuation of a sequence."

Your lecture's diagram for the decoder-only case explicitly shows:

- Token embeddings + positional encodings as input,
- A stack of masked self-attention + FFN blocks,
- Final projection to vocabulary with softmax for next-token prediction.

 Lecture 19 Transformers

If you want, next we can:

- Turn this into a **tiny text diagram** for the whole pipeline, or
- Do a **few multiple-choice questions specifically about decoder-only Transformers** to pressure-test this understanding.

Sources



After that I also want to see if GPT is able to test me with free response problems

I see, now let's move on to some free response problem, please ask me free response problem one at a time

Sounds good, let's switch to free response mode 🧠👉

Free Response Question 1 — Attention motivation + high-level picture

In the original seq2seq **encoder-decoder RNN** setup for machine translation (no attention):

1. Explain **what the bottleneck problem is** in your own words.
2. Then explain, conceptually (no equations needed), **how attention fixes this bottleneck**.
3. Use the "database" / "differentiable lookup table" view to describe the roles of **queries, keys, and values** in this seq2seq-with-attention setting (who is Q, who is K, who is V?).

You can write a short paragraph or a few bullet points.



I notice that GPT is also a great FRQ writer, it can write FRQ problems for a concept

In the original seq2seq RNN, the entire source sentence has to be squashed into a single fixed-size context vector, so as sentences get longer the encoder is forced to "stuff" more and more information into that one vector, creating a severe information bottleneck. Attention fixes this by letting the decoder look back at all encoder hidden states at each decoding step and dynamically pick (via weights) which parts of the source to focus on, instead of relying only on that single compressed vector. In the database view, the decoder hidden state at the current time step is the query, the encoder hidden states over all source positions are the keys, and those same encoder hidden states (or a transformed version of them) are the values that get mixed together into the context the decoder uses to produce the next word.

can you check if my understanding is correct?

Your answer is excellent — this is absolutely exam-ready ✓

Let me go line by line:

"In the original seq2seq RNN, the entire source sentence has to be squashed into a single fixed-size context vector... creating a severe information bottleneck."

When receiving user's response, it is also able to double check user's response and understanding with respect to the lecture notes "ground truth"

✓ Exactly right. That's the classic bottleneck: one vector must encode everything.

"Attention fixes this by letting the decoder look back at all encoder hidden states at each decoding step and dynamically pick (via weights) which parts of the source to focus on..."

✓ Perfect. "Look back at all encoder hidden states" + "dynamic weighting per decoding step" is exactly the key idea.

"...instead of relying only on that single compressed vector."

✓ Nice explicit contrast with the original model.

"In the database view, the decoder hidden state at the current time step is the query, the encoder hidden states over all source positions are the keys, and those same encoder hidden states (or a transformed version of them) are the values..."

✓ Spot-on. Using the same encoder states for both K and V (possibly after different linear transforms) matches the standard formulation.

"...that get mixed together into the context the decoder uses to produce the next word."

✓ This is the "weighted sum of values → context vector" story. Exactly right.

If I wanted to polish it *slightly* for an exam, I might add just one explicit phrase like:

- "The attention weights are computed as a similarity between the query and each key (e.g., via a dot product followed by softmax)."

But that's optional detail — conceptually, you're on point.

Want another free-response one, maybe about **self-attention vs cross-attention** or **multi-head** or **decoder-only masking**?

📄 👍 🗨️ ⬆️ ↺ ...

yes, let's do it

Nice, let's go again 🐱

Free Response Question 2 — Self-attention vs cross-attention

1. Define **self-attention** in your own words.
2. Define **cross-attention** in your own words.
3. In an **encoder–decoder Transformer** used for translation:
 - Where does **self-attention** appear? (Which part(s) of the model, and what are Q/K/V there?)
 - Where does **cross-attention** appear? (Which part, and what are Q/K/V there?)

Try to be explicit about “who is Q / who is K / who is V” in each case.



I am not 100% sure about this one, but I think self-attention means a layer where queries, keys, and values all come from the same sequence, so each position looks at (attends to) other positions in that same sequence, while cross-attention means queries come from one sequence and keys/values come from another sequence, so one sequence looks at another. In an encoder–decoder Transformer for translation, self-attention appears in every encoder layer (Q/K/V are all the encoder’s hidden states for the source sentence) and in the decoder’s masked self-attention layers (Q/K/V are all the decoder’s current-layer hidden states for the partially generated target sentence, but masked so each position only sees past positions). Cross-attention appears in the decoder’s encoder–decoder attention blocks, where the decoder’s hidden states at that layer are the queries, and the encoder’s output states (one per source token) provide the keys and values that the decoder attends over to incorporate information from the source sentence.

This is really good, especially for something you said you’re not 100% sure about 😊
Let’s check it piece by piece.

1. Your definition of self-attention

"self-attention means a layer where queries, keys, and values all come from the same sequence, so each position looks at (attends to) other positions in that same sequence"

✅ This is exactly the standard definition.

If I were nitpicky, I'd just add: "(possibly also attending to itself)" to be explicit, but what you said is perfect.

2. Your definition of cross-attention

"cross-attention means queries come from one sequence and keys/values come from another sequence, so one sequence looks at another."

✅ Also exactly right.

That "one sequence looks at another" phrase is a very good way to remember it.

3. Where they appear in an encoder–decoder Transformer

You wrote:

"self-attention appears in every encoder layer (Q/K/V are all the encoder's hidden states for the source sentence)"

✅ Correct.

- In each encoder layer:
 - $Q = K = V$ = current encoder hidden states for the **source** sentence.

"...and in the decoder's masked self-attention layers (Q/K/V are all the decoder's current-layer hidden states for the partially generated target sentence, but masked so each position only sees past positions)."

✅ Correct again.

- In each **decoder** self-attention block:
 - $Q = K = V$ = current decoder hidden states for the **target** side.

- A **causal mask** is applied so position t only sees positions $\leq t$.

"Cross-attention appears in the decoder's encoder–decoder attention blocks, where the decoder's hidden states at that layer are the queries, and the encoder's output states (one per source token) provide the keys and values..."

✅ Perfect.

More explicitly:

- At decoder layer ℓ , cross-attention:
 - Queries Q : decoder hidden states (from that layer, after self-attention).
 - Keys/Values K, V : encoder output states (final encoder layer's outputs) for the **source** tokens.

This is exactly what the lecture notes do.

Verdict

You're **fully correct** on:

- Conceptual definitions of self-attention and cross-attention
- The exact locations where they appear in an encoder–decoder Transformer
- Who plays the role of Q/K/V in each case

This is a strong exam-style answer already.

Want to try another free-response one on:

- **Multi-head attention** (what's the point structurally and intuitively?), or
- **Positional encodings** (why needed + one concrete method, like sinusoids or RoPE?), or
- **Decoder-only masked attention** (derive the mask behavior for position t)?



let's do one with Positional encodings

Nice choice — positions are where Transformers secretly cheat 🙄

Free Response Question 3 — Positional Encodings

Answer in a few sentences or short bullets:

1. **Why are positional encodings (or some form of positional information) necessary in a Transformer?**
(Hint: think about what happens if you just feed token embeddings into self-attention with no positions.)
2. Briefly describe **one specific way** to add positional information to a Transformer, based on the lecture:
 - Either **sinusoidal absolute positional encodings**,
 - Or **learned positional embeddings**,
 - Or **relative position bias / RoPE-style rotary encodings** (pick one you're most comfortable with).
3. In your chosen method, **where** is the positional information injected into the computation?
For example: added to embeddings, added to attention scores, mixed into Q/K, etc.

Your turn 😊



Here I want to see if GPT can also explain the “answer key” to the user when user has no clue what the answer is

umm, I actually don't know the answer to these questions, can you tell me the answer and explain them to me one by one?

Totally fair — this is one of those “seems obvious later” things. Let's go through them slowly and cleanly.

1. Why do Transformers need positional encodings?

Think about what happens if you **don't** give the model any positional information:

- Input is a sequence of token embeddings:

$$X = \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_T \end{bmatrix}$$

- Self-attention computes:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

and then $\text{softmax}(QK^\top)V$.

If you **permute the rows** of X (shuffle the tokens), the same permutation is applied to Q, K, V , and the attention matrix $QK^\top$ just gets its rows and columns permuted in the same way. The whole computation is **permutation-equivariant**:

If you reorder the tokens, the outputs are just reordered in the same way.

That means the model **cannot tell "dog bites man" from "man bites dog"** — it only sees a *multiset* of token embeddings, with no notion of order.

So:

Positional encodings are necessary because self-attention alone has no sense of order.

They inject information about "this is position 1, this is position 2, ..." so the model can distinguish different permutations of the same tokens.

That's the core conceptual answer.

2. One specific method: sinusoidal positional encodings

Let's pick **sinusoidal absolute positional encodings** (the original Transformer method).

Idea:

For each position $t = 0, 1, 2, \dots$, we create a vector $p_t \in \mathbb{R}^d$ using sinusoids with different frequencies:

- For even dimensions $2i$:

$$p_t[2i] = \sin\left(\frac{t}{10000^{2i/d}}\right)$$

- For odd dimensions $2i + 1$:

$$p_t[2i + 1] = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

Key properties they like:

- Each position has a **unique** pattern of sines and cosines.
- Relative positions can be expressed nicely (shifts in t become phase shifts), which helps the model reason about relative distances.
- No learned parameters for position — the formula is fixed.

That's enough detail for an exam-level description:

"Each position gets a deterministic vector built from sinusoids of different frequencies."

3. Where is positional information injected?

For sinusoidal encodings, the injection is **very simple**:

1. You have token embedding e_t for position t .
2. You compute positional encoding p_t .
3. You **add** them:

$$h_t^{(0)} = e_t + p_t$$

So the input to the **first Transformer layer** is:

$$H^{(0)} = \begin{bmatrix} e_1 + p_1 \\ e_2 + p_2 \\ \vdots \\ e_T + p_T \end{bmatrix}$$

Then all the later computation (Q/K/V, attention, FFNs) operates on these position-aware vectors. The model can now do attention that depends both on *what* the token is and *where* it sits in the sequence.

So a clean exam answer:

We compute a positional encoding vector p_t for each position t , and we **add** it to the token embedding e_t . The sum $e_t + p_t$ is used as the input to the Transformer layers. This way, self-attention sees both token identity and position.

If you want a *tiny* bonus peek beyond sinusoids (since your lecture also mentions these):

- **Learned positional embeddings:** instead of sin/cos formulas, you just have a lookup table $P \in \mathbb{R}^{T_{\max} \times d}$ and learn p_t like word embeddings. Still added: $h_t^{(0)} = e_t + p_t$.
- **Relative position bias / RoPE:** instead of adding to embeddings, they inject position into the **attention scores** or into Q/K themselves (e.g., rotate Q/K in a 2D plane based on the position). But the big idea is the same: "give the model enough info to know which token is earlier / later / how far apart."

If you'd like, we can now:

- Have you try your own short answer to this same 3-part positional encoding question,
- or
- Move on to another free-response topic (e.g., multi-head intuition, or decoder-only masked attention).



nice, I love the explanation, let's move on to another free-response topic

Awesome, let's level it up a bit 😎

Free Response Question 4 — Multi-Head Attention: What's the Point?

Answer in a few sentences / bullet points:

1. What problem or limitation would we have if we only used a single attention head instead of multi-head attention?

(Think intuitively about what one head can "focus on" vs several heads.)

2. Conceptually describe how multi-head attention works:

- What happens to Q, K, V for each head?
- What do we do with the outputs of all the heads?

3. Give one concrete example of different "types of patterns" or "types of information" that different heads might specialize in (even informally, e.g., "one head might track subject-verb relationships, another might focus on long-range dependencies," etc.).

Your turn 😊



With a single attention head, the model can only focus on one pattern / one notion of similarity at a time, so it struggles to simultaneously capture different kinds of relationships (e.g., local vs long-range, syntax vs semantics).

In multi-head attention, we linearly project Q/K/V into different subspaces for each head, run separate attention in each head, then concatenate all head outputs and pass them through another linear layer; for example, one head might specialize in subject-verb agreement, another in coreference (which pronoun refers to which noun), and another in very long-range context like topic or discourse structure.

This is a very strong answer — nicely phrased and totally consistent with the lecture ✅

Let me go through your points:

"With a single attention head, the model can only focus on one pattern / one notion of similarity at a time, so it struggles to simultaneously capture different kinds of relationships (e.g., local vs long-range, syntax vs semantics)."

- ✓ Exactly the right intuition.
- A single head produces **one** attention pattern per layer — one "view" of the sequence.
- Multi-head lets the model learn **different similarity metrics** in different subspaces, so it can capture multiple patterns *in parallel*.

"In multi-head attention, we linearly project Q/K/V into different subspaces for each head, run separate attention in each head, then concatenate all head outputs and pass them through another linear layer..."

- ✓ This is literally the algorithm:
 - For head h :

$$Q_h = XW_Q^{(h)}, K_h = XW_K^{(h)}, V_h = XW_V^{(h)}$$
 Then $\text{head}_h = \text{Attn}(Q_h, K_h, V_h)$.
 - Stack: $[\text{head}_1, \dots, \text{head}_H]$.
 - Project back with W_O .

"...for example, one head might specialize in subject–verb agreement, another in coreference (which pronoun refers to which noun), and another in very long-range context like topic or discourse structure."

- ✓ This is exactly the kind of example people use to describe what different heads *often* learn.
- You're showing you understand the **qualitative behavior**, not just the formula.

Overall: this is absolutely exam-ready already. I wouldn't change anything, except maybe explicitly mention that each head usually has **smaller dimensionality** (e.g., $d_{\text{head}} = d_{\text{model}}/H$) so total cost stays similar — but that's extra detail, not essential.

Overall, GPT did a great job in testing my understanding of the material covered in lecture 18, and 19. It is able to generate both MCQ and FRQ questions and give great explanation when user gets stuck